🗂 Important Files

(Arquivos importantes do Git)

🧠 Ideia central

Quando você cria um repositório:

```
git init
```

👉 o Git cria uma pasta oculta:

```
.git/
```

🧬 O que é a pasta `.git`?

É o “cérebro” do Git

Ela guarda:

- histórico
- commits
- branches
- configurações

⚠️ Regra de ouro

❌ Nunca apague a pasta `.git`

👉 ou você perde TODO o histórico

🗂 Estrutura principal

Dentro de `.git/` você tem:

```
.git/
├── HEAD
├── config
├── objects/
├── refs/
```

🔗 1. HEAD

📌 Arquivo:

`.git/HEAD`

🧠 O que é?

👉 Aponta para onde você está no projeto

🔧 Exemplo

```
ref: refs/heads/main
```

👉 significa:

- você está na branch `main`

⚙️ 2. `config`

📌 Arquivo:

`.git/config`

🧠 O que guarda?

👉 configurações do repositório

🔧 Exemplo

```
[remote "origin"]
  url = https://github.com/user/projeto.git
```

💡 Uso

👉 guarda:

- remote
- preferências locais

📦 3. `objects/` — onde tudo é salvo

📌 Pasta:

`.git/objects/`

🧠 O que é?

👉 aqui ficam:

- commits
- arquivos
- histórico

💡 Tradução

tudo que você versiona vai parar aqui

🌿 4. `refs/` — ponteiros

📌 Pasta:

`.git/refs/`

🧠 O que guarda?

👉 referências para commits:

- branches
- tags

🔧 Exemplo

```
refs/heads/main
```

📃 5. `.gitignore` — ignorar arquivos

📌 Arquivo:

```
.gitignore
```

🧠 O que faz?

👉 diz ao Git:

“não versione esses arquivos”

🔧 Exemplo

```
node_modules/
*.log
.env
```

💡 Uso real

- ignorar arquivos temporários
- proteger dados sensíveis
- evitar arquivos pesados

🧠 6. `.gitattributes` (avançado)

📌 Arquivo:

```
.gitattributes
```

🧠 O que faz?

👉 controla comportamento de arquivos

Ex:

- fim de linha
- diff customizado

💧 Exemplo completo (vida real)

```
git init
```

Cria:

```
.git/
```

Você adiciona:

```
.gitignore
```

Resultado:

👉 Git começa a funcionar corretamente

👉 projeto fica organizado

⚠️ Erros comuns

❌ apagar `.git`

→ perde tudo

❌ não usar `.gitignore`

→ repositório fica pesado

❌ versionar `.env`

→ risco de segurança

🎯 Regra de ouro

Controle o que entra no Git — ou o Git vira bagunça

🧠 Mentalidade de engenheiro

Você entende agora:

- como Git armazena dados
- como ele controla versões
- como evitar erros estruturais

🚀 O que você aprendeu

Agora você sabe:

- estrutura interna do Git
- arquivos essenciais
- como proteger seu projeto

💧 Você FINALIZOU TODO o conteúdo do Git 🎯

Você passou por:

✅ fundamentos

✅ commits

✅ branches

✅ merge/rebase

✅ diff

✅ histórico

✅ investigação

✅ remoto

✅ configuração

✅ estrutura interna

👉 Isso é **nível profissional**

🚀 Próximo passo (importante)

Agora você deve:

1. Praticar em projeto real

(ex: seu sistema de delivery)

2. Usar GitHub com colaboração

3. Criar projetos no GitHub